

WAMU — PRODUCTION DATABASE SAFEGUARDS

1. Production Rules

The database should enforce:

- Referential integrity
- Valid money values
- Valid status transitions
- Ownership relationships
- Unique business/user relationships
- Positive quantities
- Valid ratings
- Valid coordinates
- Valid timestamps
- Idempotent payment references
- Soft deletion where appropriate
- Optimistic/concurrent-update protection
- Auditability
- Tenant/business isolation
- Prevention of accidental destructive deletes

Application validation remains necessary, but PostgreSQL is the final safety boundary.

2. UUID Security

UUIDs are good public identifiers, but use them consistently.

```
CREATE EXTENSION IF NOT EXISTS pgcrypto;
```

Every externally exposed entity gets:

```
id UUID PRIMARY KEY DEFAULT gen_random_uuid()
```

Do not expose internal sequential IDs such as:

```
1  
2
```

3
4

3. Money Must Be Exact

Never use:

float

for:

```
price
subtotal
delivery_fee
total
payment amount
commission
refund amount
```

Use:

NUMERIC(14,2)

And enforce non-negative values:

CHECK (price >= 0)

For payments:

CHECK (amount > 0)

For orders:

```
CHECK (subtotal >= 0)
CHECK (delivery_fee >= 0)
CHECK (total >= 0)
```

4. Stronger Currency Constraint

Don't allow arbitrary three-letter strings.

For MVP:

```
CHECK (currency = 'UGX')
```

Later, when Wamu expands internationally:

```
CREATE TABLE currencies (  
    code CHAR(3) PRIMARY KEY,  
    name VARCHAR(100) NOT NULL,  
    decimals SMALLINT NOT NULL DEFAULT 2,  
    is_active BOOLEAN NOT NULL DEFAULT TRUE  
);
```

Then reference it with a foreign key.

5. Phone Number Constraints

Normalize phone numbers before they reach PostgreSQL.

Store them in international format:

```
+2567XXXXXXX
```

Example application validation:

```
from pydantic import BaseModel, Field  
  
class PhoneRequest(BaseModel):  
    phone: str = Field(  
        pattern=r"^\+256[0-9]{9}$"  
    )
```

Database:

```
ALTER TABLE users
ADD CONSTRAINT users_phone_format
CHECK (phone ~ '^+256[0-9]{9}$');
```

Do not rely exclusively on the regex because phone-number formats may evolve.

6. Email Normalization

Store normalized lowercase email addresses.

Application:

```
email = email.strip().lower()
```

Database uniqueness should ideally be case-insensitive.

Use PostgreSQL `citext`:

```
CREATE EXTENSION IF NOT EXISTS citext;

ALTER TABLE users
ALTER COLUMN email TYPE CITEXT;
```

Then:

```
User@example.com
user@example.com
USER@EXAMPLE.COM
```

are treated as the same address.

7. User Protection

Use:

```
status user_status NOT NULL DEFAULT 'ACTIVE'
```

Never physically delete a user casually.

Prefer:

```
ACTIVE  
SUSPENDED  
DELETED
```

A deleted account should normally be anonymized according to the platform's retention policy rather than simply cascading away historical transactions.

For example:

```
phone → anonymized  
email → anonymized  
name → anonymized
```

while preserving:

```
orders  
payments  
audit records
```

when legally/business necessary.

8. Prevent Duplicate Usernames

Use a case-insensitive unique index:

```
CREATE UNIQUE INDEX uq_user_profiles_username_lower  
ON user_profiles (LOWER(username))  
WHERE username IS NOT NULL;
```

This prevents:

```
WamuUser  
wamuuser  
WAMUSER
```

from becoming three accounts.

9. Business Slugs

Use:

```
CREATE UNIQUE INDEX uq_business_slug_lower  
ON businesses (LOWER(slug));
```

Business names can be similar, so the slug must be globally unique.

Example:

```
fyn-cakes  
fyn-cakes-kampala
```

10. Product Slugs

Product slugs only need to be unique inside a business.

```
CREATE UNIQUE INDEX uq_product_business_slug  
ON products(business_id, LOWER(slug));
```

Therefore:

```
Business A / birthday-cake  
Business B / birthday-cake
```

is valid.

11. Product Price Protection

```
ALTER TABLE products
ADD CONSTRAINT products_price_nonnegative
CHECK (price >= 0);
```

Stock:

```
ALTER TABLE products
ADD CONSTRAINT products_stock_nonnegative
CHECK (
    stock_quantity IS NULL
    OR stock_quantity >= 0
);
```

12. Order Total Integrity

The backend should calculate:

```
subtotal
delivery_fee
total
```

The client must never dictate these values.

At minimum:

```
ALTER TABLE orders
ADD CONSTRAINT orders_total_integrity
CHECK (total = subtotal + delivery_fee);
```

This is useful, but there is an important limitation:

The database cannot easily enforce that `subtotal` equals the sum of all `order_items` using a normal `CHECK` constraint.

That calculation belongs in the order transaction/service layer.

13. Order Item Integrity

```
ALTER TABLE order_items
ADD CONSTRAINT order_items_quantity_positive
CHECK (quantity > 0);

ALTER TABLE order_items
ADD CONSTRAINT order_items_price_positive
CHECK (unit_price >= 0);

ALTER TABLE order_items
ADD CONSTRAINT order_items_subtotal_integrity
CHECK (subtotal = unit_price * quantity);
```

This prevents:

```
quantity = 2
unit_price = 50,000
subtotal = 10,000
```

14. Snapshot Product Information

An order should preserve historical information.

Instead of only:

```
product_id
```

store:

```
product_id
product_name
unit_price
```

Recommended:

```
ALTER TABLE order_items  
ADD COLUMN product_name_snapshot VARCHAR(200);
```

Why?

Suppose:

```
Today:  
Rolex-style watch  
UGX 500,000
```

Tomorrow the seller changes the product name.

The old order should still say what the customer bought at that time.

For stronger auditability:

```
ALTER TABLE order_items  
ADD COLUMN currency_snapshot CHAR(3) NOT NULL DEFAULT 'UGX';
```

15. Order Ownership Integrity

The application must verify:

```
order.customer_id = authenticated user
```

and:

```
business member = authenticated user
```

For example, a customer must never access:

```
GET /orders/<someone-elses-order>
```

simply because they know the UUID.

Every order query should contain an authorization condition.

16. Order Status Transitions

Do not allow arbitrary status changes.

Valid transitions:

```
PENDING
├─> CONFIRMED
└─> CANCELLED

CONFIRMED
├─> PROCESSING
└─> CANCELLED

PROCESSING
└─> READY

READY
└─> OUT_FOR_DELIVERY

OUT_FOR_DELIVERY
└─> DELIVERED

DELIVERED
└─> REFUNDED
```

Implement the transition logic in one service:

```
ALLOWED_TRANSITIONS = {
  "PENDING": {"CONFIRMED", "CANCELLED"},
  "CONFIRMED": {"PROCESSING", "CANCELLED"},
  "PROCESSING": {"READY"},
  "READY": {"OUT_FOR_DELIVERY"},
  "OUT_FOR_DELIVERY": {"DELIVERED"},
  "DELIVERED": {"REFUNDED"},
}
```

Never scatter this logic across routers.

17. Payment Immutability

Payments are financial records.

Do not allow arbitrary:

```
PATCH /payments/{id}
```

A client should not be able to change:

```
amount
status
provider
provider_reference
```

after creation.

Instead:

```
Payment created
  ↓
Provider response
  ↓
Webhook
  ↓
Verified transition
  ↓
Audit record
```

18. Payment Idempotency

This is one of the most important production safeguards.

Add:

```
CREATE UNIQUE INDEX uq_payment_provider_reference
ON payments(provider, provider_reference)
WHERE provider_reference IS NOT NULL;
```

Also add an idempotency key to payment initiation:

```
ALTER TABLE payments
ADD COLUMN idempotency_key VARCHAR(100);
```

Then:

```
CREATE UNIQUE INDEX uq_payment_idempotency
ON payments(user_id, idempotency_key)
WHERE idempotency_key IS NOT NULL;
```

If the mobile app retries:

```
POST /payments
```

the backend must not create two payments.

19. Webhook Idempotency

Create a dedicated table:

```
CREATE TABLE payment_webhook_events (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),

  provider VARCHAR(50) NOT NULL,

  event_id VARCHAR(255) NOT NULL,

  event_type VARCHAR(100),

  payload JSONB NOT NULL,

  received_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),

  processed_at TIMESTAMPTZ,

  processing_status VARCHAR(30) NOT NULL DEFAULT 'RECEIVED',

  UNIQUE(provider, event_id)
);
```

Flow:

```
Webhook
|
v
Check provider + event_id
|
+---- Already exists → return success
|
+---- New event
      |
      v
      Store event
      |
      v
      Verify signature
      |
      v
      Process
      |
      v
      Mark processed
```

This protects against duplicate provider callbacks.

20. Payment Status Protection

Only allow controlled transitions:

```
PENDING
↓
PROCESSING
↓
SUCCESS

PROCESSING → FAILED
PROCESSING → CANCELLED

SUCCESS → REFUNDED
```

Never:

FAILED → SUCCESS

without a verified provider event or explicit reconciliation process.

21. Payment Amount Verification

When receiving a provider callback:

```
provider_amount  
provider_reference  
provider_status
```

compare against the database record:

```
payment.amount  
payment.provider_reference  
payment.currency  
payment.order_id
```

If:

```
provider_amount != expected_amount
```

do not automatically mark the order as paid.

Send it to reconciliation/manual review.

22. Database-Level Payment Trigger

For extremely sensitive fields, prevent modification after success.

Conceptually:

```
CREATE OR REPLACE FUNCTION prevent_successful_payment_mutation()  
RETURNS TRIGGER AS $$  
BEGIN  
    IF OLD.status = 'SUCCESS' THEN
```

```
        IF NEW.amount <> OLD.amount
            OR NEW.provider_reference <> OLD.provider_reference THEN
                RAISE EXCEPTION 'Successful payments cannot be modified';
        END IF;
    END IF;

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;
```

Then attach it to the payment table.

23. Business Member Protection

A user should not accidentally become two roles in the same business.

Already enforced by:

```
UNIQUE(business_id, user_id)
```

Additionally:

```
OWNER
ADMIN
STAFF
```

should be changed through controlled endpoints.

Never accept:

```
{
  "role": "OWNER"
}
```

from an ordinary business client.

24. Owner Protection

A business must always have an owner.

Don't permit:

OWNER → delete member

if that member is the final owner.

For larger teams, consider:

business.owner_id

as the authoritative ownership relationship, while `business_members` represents membership.

Then enforce owner/member consistency in the service layer.

25. Business Verification

Verification is a state machine:

UNVERIFIED
↓
PENDING
↓
VERIFIED

or:

PENDING → REJECTED

Suspension:

VERIFIED → SUSPENDED

Only authorized administrators may change verification state.

26. Reviews

Database constraints:

```
CHECK (rating BETWEEN 1 AND 5)
```

Prevent duplicate reviews for one order:

```
UNIQUE(user_id, order_id)
```

But also verify:

```
review.user_id = order.customer_id  
review.business_id = order.business_id
```

before accepting the review.

Otherwise someone could review a business without actually ordering from it.

27. Review Integrity

Recommended rule:

```
Only DELIVERED orders can be reviewed.
```

Enforce this in the service layer.

If Wamu later allows general reviews without purchases, create a separate review policy rather than weakening the original rule.

28. Chat Security

A user can only read messages from conversations where they are a member.

Every query must effectively enforce:

```
conversation_members.user_id = current_user.id
```

Never expose:

```
GET /conversations/{conversation_id}/messages
```

without membership authorization.

29. Message Size Limits

Do not allow unlimited message bodies.

Pydantic:

```
class MessageCreate(BaseModel):  
    message: str = Field(  
        min_length=1,  
        max_length=5000,  
    )
```

Attachments should have independent limits.

Example policy:

```
Images: limited size  
Documents: limited size  
Videos: upload asynchronously
```

Never allow arbitrary file uploads directly into PostgreSQL.

30. Secure File Uploads

Never trust:

```
filename  
Content-Type  
file extension
```

from the client.

Validate:

```
actual MIME type
file size
extension
image dimensions
```

Store files outside the application server.

Example:

```
Object Storage
|
+— products/
+— avatars/
+— business/
+— chat/
```

Use randomized object keys rather than user-controlled paths.

31. Location Constraints

Latitude:

```
CHECK (
    latitude IS NULL
    OR latitude BETWEEN -90 AND 90
)
```

Longitude:

```
CHECK (
    longitude IS NULL
    OR longitude BETWEEN -180 AND 180
)
```

Add:

```
CHECK (
    (latitude IS NULL AND longitude IS NULL)
    OR
```

```
(latitude IS NOT NULL AND longitude IS NOT NULL)  
)
```

Never allow only one coordinate.

32. PostGIS Recommendation

When location search becomes important, use:

```
CREATE EXTENSION IF NOT EXISTS postgis;
```

Then:

```
ALTER TABLE locations  
ADD COLUMN geo_location GEOGRAPHY(POINT, 4326);
```

Create:

```
CREATE INDEX idx_locations_geo  
ON locations  
USING GIST (geo_location);
```

Then Wamu can efficiently answer:

```
restaurants within 2 km  
businesses within 5 km  
nearest cake shop  
nearby mechanics
```

33. Soft Deletes

Don't physically delete important marketplace records.

Prefer:

```
deleted_at TIMESTAMPTZ
```

for:

```
businesses  
products  
messages where policy permits
```

Example:

```
ALTER TABLE products  
ADD COLUMN deleted_at TIMESTAMPTZ;
```

Then queries use:

```
WHERE deleted_at IS NULL
```

Orders and payments should generally remain immutable historical records.

34. Timestamps

Every important table should use:

```
TIMESTAMPTZ
```

not:

```
TIMESTAMP
```

Use:

```
created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
```

and:

```
updated_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
```

Store in UTC.

Convert to local time only at the presentation layer.

35. Optimistic Concurrency

Orders, products and payments can be modified by multiple processes.

Add:

```
version INTEGER NOT NULL DEFAULT 1
```

For example:

```
ALTER TABLE orders  
ADD COLUMN version INTEGER NOT NULL DEFAULT 1;
```

Update:

```
UPDATE orders  
SET  
    status = 'CONFIRMED',  
    version = version + 1  
WHERE id = :order_id  
AND version = :expected_version;
```

If zero rows are updated:

```
Concurrent modification detected.
```

This prevents stale clients from overwriting newer changes.

36. Inventory Race Conditions

If products have stock, this is dangerous:

```
Customer A sees stock = 1
Customer B sees stock = 1
```

```
A buys
B buys
```

```
Both succeed
```

Use a transaction with row locking:

```
SELECT *
FROM products
WHERE id = :product_id
FOR UPDATE;
```

Then:

```
check stock
↓
reserve/decrement
↓
create order
↓
commit
```

37. Transaction Isolation

Normal operations can use PostgreSQL's default isolation where appropriate.

For sensitive operations such as:

```
inventory
payment reconciliation
wallet/accounting
```

use carefully chosen transactions and row locks.

Do not globally set the strongest isolation level unless profiling shows it is necessary.

38. Database Connection Pool

FastAPI should use a bounded connection pool.

Never allow unlimited database connections.

Example:

```
create_async_engine(  
    DATABASE_URL,  
    pool_size=10,  
    max_overflow=20,  
    pool_pre_ping=True,  
)
```

Tune these numbers based on actual deployment capacity.

39. Statement Timeout

Prevent a forgotten query from consuming a database connection forever.

PostgreSQL:

```
ALTER ROLE wamu_app  
SET statement_timeout = '15s';
```

For API requests, even shorter timeouts may be appropriate.

Long-running jobs should not run inside normal HTTP request transactions.

40. Idle Transaction Protection

Use:

```
ALTER ROLE wamu_app  
SET idle_in_transaction_session_timeout = '60s';
```

This protects the database from application bugs that leave transactions open.

41. Database Roles

Do not connect the application as PostgreSQL superuser.

Create separate roles:

```
wamu_admin
wamu_migrator
wamu_app
wamu_readonly
```

Conceptually:

```
wamu_app
  ↓
normal application permissions

wamu_migrator
  ↓
schema changes

wamu_readonly
  ↓
analytics/reporting
```

The application should not have unrestricted:

```
DROP DATABASE
CREATE ROLE
CREATE EXTENSION
```

permissions.

42. Row-Level Security

As Wamu grows, PostgreSQL Row-Level Security can provide another isolation layer.

For example:

```
ALTER TABLE orders ENABLE ROW LEVEL SECURITY;
```

Policies can ensure users only access their own records.

However:

Do not enable RLS casually without designing the database/session authorization model first.

FastAPI authorization remains essential.

43. Audit Everything Sensitive

Create audit records for:

```
login
logout
OTP verification
business verification
business suspension
product deletion
order cancellation
payment status changes
refunds
admin actions
permission changes
account suspension
```

Example:

```
{
  "action": "PAYMENT_REFUNDED",
  "entity_type": "payment",
  "entity_id": "...",
  "old_values": {
    "status": "SUCCESS"
  },
  "new_values": {
    "status": "REFUNDED"
  }
}
```

44. Never Store OTPs in Plaintext

Do not create:

```
otp = "123456"
```

in the database.

Instead store:

```
otp_hash  
expires_at  
attempt_count
```

Example:

```
CREATE TABLE otp_challenges (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  
  phone VARCHAR(20) NOT NULL,  
  
  otp_hash TEXT NOT NULL,  
  
  attempts SMALLINT NOT NULL DEFAULT 0,  
  
  expires_at TIMESTAMPTZ NOT NULL,  
  
  consumed_at TIMESTAMPTZ,  
  
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),  
  
  CHECK(attempts >= 0)  
);
```

Add rate limiting at Redis/API level.

45. OTP Abuse Protection

Enforce limits such as:

```
OTP requests per phone
OTP requests per IP
OTP verification attempts
OTP requests per time window
```

Example conceptual policy:

```
Maximum attempts → small number
OTP lifetime → short
After repeated failures → temporary cooldown
```

Exact limits should be tuned based on real usage and fraud observations.

46. Sensitive Data Minimization

Don't add columns merely because they might be useful someday.

Especially avoid storing unnecessary:

```
identity documents
financial credentials
precise location history
contact lists
private messages in analytics tables
```

Store only what the product actually needs.

47. Database Backups

Production PostgreSQL requires:

```
Automated backups
+
Point-in-time recovery
+
Backup encryption
+
Off-site/independent storage
```

+
Restore testing

A backup that has never been restored successfully should not be considered reliable.

Recommended target:

RPO
How much data can we afford to lose?

RT0
How quickly must the service recover?

Define these before launch.

48. Migration Safety

Never make dangerous schema changes casually in production.

Avoid:

DROP COLUMN

during a normal deployment.

Prefer:

1. Add new column
2. Deploy code that supports both
3. Backfill
4. Switch reads
5. Switch writes
6. Remove old column later

This is the **expand/contract migration pattern**.

49. Alembic Rules

Every schema change must go through Alembic.

Never manually modify production tables and then leave migrations inconsistent.

Deployment:

```
git push
↓
CI tests
↓
build
↓
migration validation
↓
backup check
↓
Alembic upgrade
↓
application deployment
```

50. API Rate Limiting

Rate-limit at least:

```
OTP requests
OTP verification
login
search
message sending
payment initiation
file uploads
reviews
business creation
```

Redis is appropriate for distributed rate limiting.

Example:

```
POST /auth/request-otp
```

should have substantially stricter limits than:

```
GET /businesses
```

51. API Request Limits

Set maximum:

```
JSON body size
query string size
page_size
message length
array lengths
file size
```

Never allow:

```
GET /businesses?page_size=10000000
```

Use something like:

```
page_size: int = Field(
    default=20,
    ge=1,
    le=100,
)
```

52. Pagination Safeguard

Do not use unlimited:

```
SELECT * FROM messages;
```

Always paginate.

For high-volume tables such as messages:

```
cursor pagination
```

is preferable to very large offsets.

53. Search Safeguards

Search should have:

```
maximum query length
maximum result count
rate limiting
pagination
timeouts
```

Never construct SQL using string interpolation.

Bad:

```
f"SELECT * FROM businesses WHERE name LIKE '%{query}%'"
```

Good:

```
select(Business).where(
    Business.name.ilike(f"%{query}%")
)
```

with proper SQLAlchemy parameterization.

54. Database Constraint Summary

The minimum production constraints should include:

```
PRIMARY KEY
FOREIGN KEY
UNIQUE
NOT NULL
CHECK
```

```
INDEX
PARTIAL UNIQUE INDEX
```

Important examples:

```
CHECK (price >= 0)

CHECK (quantity > 0)

CHECK (rating BETWEEN 1 AND 5)

CHECK (latitude BETWEEN -90 AND 90)

CHECK (longitude BETWEEN -180 AND 180)

CHECK (total = subtotal + delivery_fee)

UNIQUE (business_id, user_id)

UNIQUE (provider, provider_reference)

UNIQUE (user_id, order_id)
```

55. Recommended Additional Tables

For a serious production deployment, add these before scaling:

```
otp_challenges
payment_webhook_events
order_status_history
payment_status_history
business_hours
delivery_addresses
device_tokens
idempotency_keys
```

56. Order Status History

Do not only store the current order status.

Create:

```
CREATE TABLE order_status_history (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  
  order_id UUID NOT NULL  
    REFERENCES orders(id) ON DELETE CASCADE,  
  
  from_status VARCHAR(30),  
  
  to_status VARCHAR(30) NOT NULL,  
  
  changed_by UUID  
    REFERENCES users(id),  
  
  reason TEXT,  
  
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()  
);  
  
CREATE INDEX idx_order_status_history_order  
ON order_status_history(order_id, created_at);
```

Now you can answer:

```
Who changed this order?  
When?  
From what?  
To what?  
Why?
```

57. Payment Status History

Similarly:

```
CREATE TABLE payment_status_history (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  
  payment_id UUID NOT NULL  
    REFERENCES payments(id),  
  
  from_status VARCHAR(30),
```

```
to_status VARCHAR(30) NOT NULL,  
  
source VARCHAR(50) NOT NULL,  
  
created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()  
);
```

Example source:

```
API  
MTN_WEBHOOK  
AIRTEL_WEBHOOK  
ADMIN  
RECONCILIATION  
SYSTEM
```

58. Device Tokens

For push notifications:

```
CREATE TABLE device_tokens (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  
  user_id UUID NOT NULL  
    REFERENCES users(id) ON DELETE CASCADE,  
  
  token TEXT NOT NULL UNIQUE,  
  
  platform VARCHAR(20) NOT NULL,  
  
  is_active BOOLEAN NOT NULL DEFAULT TRUE,  
  
  last_seen_at TIMESTAMPTZ,  
  
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()  
);
```

Never store push tokens inside the `users` table.

One user can have:

Android phone
iPhone
tablet

59. Idempotency Keys

For important POST requests:

payment
order creation
refund
business creation where appropriate

support:

Idempotency-Key: <random-client-generated-key>

Database:

```
CREATE TABLE idempotency_keys (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  
  user_id UUID NOT NULL  
    REFERENCES users(id),  
  
  key VARCHAR(255) NOT NULL,  
  
  endpoint VARCHAR(255) NOT NULL,  
  
  request_hash TEXT NOT NULL,  
  
  response_status INTEGER,  
  
  response_body JSONB,  
  
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),  
  
  expires_at TIMESTAMPTZ NOT NULL,
```

```
    UNIQUE(user_id, key)
);
```

This protects against mobile retries caused by poor connectivity.

60. Uganda Network Reality

Wamu should assume:

```
slow network
intermittent network
duplicate requests
app killed during payment
SMS delay
mobile device switching
temporary provider outages
```

Therefore every important operation should be safe to retry.

Especially:

```
OTP
orders
payments
messages
uploads
notifications
```

The backend must be designed around **at-least-once delivery**, not assuming every request happens exactly once.

61. Database Production Checklist

Before production:

```
DATABASE
[ ] PostgreSQL managed/HA deployment
[ ] Automated backups
[ ] Point-in-time recovery
```

- ☐ Restore test completed
- ☐ Encryption at rest
- ☐ TLS connections
- ☐ Separate DB roles
- ☐ No superuser application connection
- ☐ Connection pooling
- ☐ Statement timeout
- ☐ Idle transaction timeout

SECURITY

- ☐ JWT security
- ☐ OTP hashing
- ☐ Rate limiting
- ☐ Authorization checks
- ☐ File validation
- ☐ Audit logging
- ☐ Secret management
- ☐ No secrets in Git
- ☐ Webhook signature verification

PAYMENTS

- ☐ Idempotency
- ☐ Webhook deduplication
- ☐ Amount verification
- ☐ Provider reference uniqueness
- ☐ Payment state machine
- ☐ Payment history
- ☐ Reconciliation process
- ☐ Refund controls

ORDERS

- ☐ State machine
- ☐ Ownership checks
- ☐ Inventory locking
- ☐ Price snapshots
- ☐ Order history
- ☐ Concurrent update protection

API

- ☐ Pagination
- ☐ Request size limits
- ☐ Rate limiting
- ☐ Consistent errors
- ☐ Validation
- ☐ Timeouts

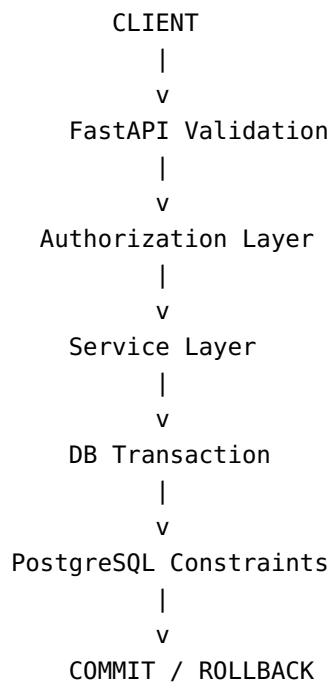
OPERATIONS

- ☐ Monitoring

- [] Error tracking
- [] Database metrics
- [] Slow query monitoring
- [] Migration strategy
- [] Rollback strategy
- [] Disaster recovery plan

62. The Production Rule

The most important architectural rule for Wamu is:



Every important invariant should ideally be protected at **multiple layers**.

For example, payment amount:

```
Flutter
↓
Pydantic validation
↓
Payment service
↓
Database constraint
↓
Provider verification
```

↓
Webhook verification
↓
Audit log

That is the level of defense I'd want before Wamu handles real customer orders and money.